

OCPP 协议框架开发使用手册

1. 框架目标

本框架基于 Spring Boot、JDK8、Maven、Lombok 和 Spring WebSocket 实现，目标是将 OCPP 协议实现与业务系统彻底解耦。业务开发者不需要直接处理 WebSocketSession、OCPP-J 数组帧、CALL/CALLRESULT/CALLERROR、uniqueId、请求响应关联、超时清理、动作路由和异常转换。

业务侧只需要关注两类入口：

1. 充电桩主动上报时，使用 @OcppActionMapping 编写业务处理方法。
2. 中心系统主动下发命令时，注入 OcppGateway 或 OcppTemplate 调用 call 方法。

2. 工程结构

```
ocpp-protocol-framework-v2
├── ocpp-core
│   ├── enums           # OCPP 版本、消息类型
│   ├── exception       # OCPP 错误码和协议异常
│   ├── protocol        # OCPP-J 帧模型和编解码器
│   ├── handler         # 动作处理器、注册表、请求上下文
│   ├── session         # 会话仓储和连接抽象
│   ├── schema          # Schema 校验扩展点
│   └── model            # OCPP 1.6 / 2.0.1 常用 DTO
├── ocpp-spring-boot-starter
│   ├── autoconfigure   # Spring Boot 自动装配
│   ├── registry        # 注解方法扫描与反射调用
│   ├── service         # OcppTemplate 主动下发命令
│   └── websocket       # 基于 spring-websocket 的传输层
└── ocpp-demo-business  # 业务接入示例
```

3. 技术选型

- JDK: 1.8。
- Spring Boot: 2.7.x。
- WebSocket: 显式使用 org.springframework:spring-websocket。
- JSON: Jackson。
- 注解简化: Lombok。
- 构建方式: Maven 多模块工程。

本框架没有使用 Netty、STOMP 或 SockJS。OCPP JSON over WebSocket 的通信形态是原生 WebSocket 文本帧，因此 starter 中使用 TextWebSocketHandler 和 WebSocketConfigurer。

4. IDEA 导入

1. 解压 ocpp-protocol-framework-v2.zip。
2. 使用 IDEA 打开根目录。
3. 等待 Maven 自动导入三个模块。
4. 安装 IDEA Lombok 插件。
5. 开启 Annotation Processing。
6. 运行 ocpp-demo-business 模块中的 OcppDemoApplication。

5. 构建、安装与依赖引入

在根目录执行：

```
mvn clean install
```

业务系统引入 starter：

```
<dependency>
  <groupId>com.charging.ocpp</groupId>
  <artifactId>ocpp-spring-boot-starter</artifactId>
  <version>1.0.0</version>
</dependency>
```

如果业务系统也使用 Lombok，建议同时安装 IDE 插件并开启注解处理。

6. 配置项

示例配置：

```
server:
  port: 8080

ocpp:
  path: /ocpp/{chargePointId}
  connection-timeout-seconds: 60
  allow-unknown-actions: false
  supported-sub-protocols:
    - ocpp1.6
    - ocpp2.0.1
```

默认 WebSocket 地址：

```
ws://localhost:8080/ocpp/{chargePointId}
```

充电桩应在握手阶段携带子协议：

- OCPP 1.6: ocpp1.6
- OCPP 2.0.1: ocpp2.0.1

7. 业务层处理充电桩上报

业务处理器示例：

```
@Component
@Slf4j
public class DemoOcppBusinessHandler {

    @OcppActionMapping(version = OcppVersion.OCPP_16, action = "BootNotification")
    public BootNotificationResponse onBoot16(OcppRequestContext context,
                                              BootNotificationRequest request) {

        log.info("收到启动通知, 桩号={}, 厂商={}",
                context.getChargePointId(), request.getChargePointVendor());

        BootNotificationResponse response = new BootNotificationResponse();
        response.setCurrentTime("2026-05-21T00:00:00Z");
        response.setInterval(300);
        response.setStatus("Accepted");
        return response;
    }
}
```

说明：

- OcppRequestContext 提供 chargePointId、sessionId、version、action、uniqueId 等上下文。
- 请求 DTO 由框架自动从 JSON payload 转换。
- 返回 DTO 由框架自动封装为 OCPP CALLRESULT。
- 抛出的协议异常或业务异常会被转换为 CALLERROR。

8. 业务层向充电桩下发命令

推荐依赖接口：

```
@Autowired
private OcppGateway ocppGateway;
```

下发 OCPP 1.6 远程启动：

```
RemoteStartTransactionRequest request = new RemoteStartTransactionRequest();
request.setConnectorId(1);
request.setIdTag("USER001");

CompletableFuture<RemoteStartTransactionResponse> future = ocppGateway.call(
    "CP001",
    OcppVersion.OCPP_16,
    "RemoteStartTransaction",
    request,
    RemoteStartTransactionResponse.class
);
```

框架会自动完成：

- 生成 uniqueId。
- 编码为 [2, uniqueId, action, payload]。

- 查找充电桩在线连接。
- 通过 Spring WebSocket 发送文本帧。
- 保存 PendingRequest。
- 收到 CALLRESULT 时完成 Future。
- 收到 CALLERROR 时完成异常。
- 超时后清理等待中的请求。

9. Lombok 使用说明

本次增强已在 DTO 和关键类中使用常见 Lombok 注解：

注解	用途
<code>@Data</code>	自动生成 getter、setter、toString、equals、hashCode。
<code>@Builder</code>	为 DTO 提供链式构建方式。
<code>@NoArgsConstructor</code>	生成无参构造器，便于 Jackson 反序列化。
<code>@AllArgsConstructor</code>	生成全参构造器，便于测试和构造对象。
<code>@Accessors(chain = true)</code>	setter 返回 this，支持链式赋值。
<code>@Slf4j</code>	自动生成日志对象，替代手写 LoggerFactory。
<code>@RequiredArgsConstructor</code>	为 final 字段生成构造器，适合 Spring 构造器注入。

10. WebSocket 实现说明

starter 模块显式依赖 spring-websocket。核心类如下：

- `OcppWebSocketConfigurer`：注册 WebSocket 端点。
- `OcppHandshakeInterceptor`：从路径中解析 chargePointId。
- `OcppWebSocketHandshakeHandler`：处理子协议协商。
- `OcppWebSocketHandler`：处理连接、文本帧、路由、响应、异常转换。
- `SpringOcppConnection`：封装 Spring WebSocketSession，向 core 暴露统一连接接口。

协议层不直接依赖业务代码；业务层也不直接依赖 WebSocketSession。

11. OCPP 1.6 和 2.0.1 兼容方式

框架通过 `OcppVersion` 区分协议版本，通过 version + action 路由到不同业务处理器。这样即使两个版本存在同名 Action，也不会路由冲突。

内置 DTO 覆盖常见动作，例如：

- OCPP 1.6: BootNotification、Authorize、Heartbeat、StartTransaction、StopTransaction、StatusNotification、MeterValues、RemoteStartTransaction、RemoteStopTransaction。

- OCPP 2.0.1: BootNotification、Authorize、Heartbeat、StatusNotification、MeterValues、TransactionEvent、RequestStartTransaction、RequestStopTransaction。

12. 如何扩展更高版本 OCPP

以扩展 OCPP 2.1 为例：

1. 在 `OcppVersion` 中新增版本枚举和值到子协议的映射。
2. 在 `model` 下新增 `v21 DTO` 包。
3. 增加 OCPP 2.1 的请求与响应 DTO。
4. 在业务层新增 `@OcppActionMapping(version = OcppVersion.OCPP_21, action = "...")`。
5. 如需要强 Schema 校验，实现 `OcppSchemaValidator`，替换默认 `NoopOcppSchemaValidator`。
6. 如需要版本级默认处理器，可在 `starter` 中新增 `DefaultOcpp21Handlers`。
7. 如 OCPP 2.1 引入新的 WebSocket 子协议名，在配置和握手处理逻辑中补充。

核心设计点是：编解码器只关心 OCPP-J 的公共帧结构；版本差异主要体现在 DTO、Action 和业务处理器中。

13. 多节点部署建议

默认 `InMemoryOcppSessionRepository` 适合单节点或本地开发。生产多节点部署时建议：

- 自定义实现 `OcppSessionRepository`。
- 将在线状态同步到 Redis 或注册中心。
- 主动下发命令时按 `chargePointId` 路由到持有连接的节点。
- 对 `PendingRequest` 进行节点内管理，不建议跨节点共享 `WebSocketSession`。

14. 安全建议

公网接入充电桩时建议：

- 使用 WSS。
- 在握手阶段校验桩编号、签名、token 或证书。
- 限制 `allowed origins`。
- 对原始报文做审计留存。
- 对敏感日志做脱敏。

15. 当前边界

本工程已实现协议框架、传输层、路由、典型 DTO 和示例业务。为了保持示例可读性，未穷尽所有 OCPP Action 的完整字段模型。如果用于生产级认证，应继续补齐完整 DTO 和 JSON Schema 校验文件。